Project Design Document — StreamOps Agent (DRAFT)

Version: DRAFT
Date: 03/10/2026
Change log: initial draft

---

1. Project Title & Version Control

- **Project title:** StreamOps Agent — LLM-powered assistant for streaming TPM workflows.
- **Version & date:** DRAFT — 03/10/2026.
- **Change log:** Initial project definition and sprint plan.

2. Project Summary (2–3 sentences)

- Build a MERN-stack prototype that ingests simulated streaming telemetry and connects to a retrieval-augmented LLM agent which detects anomalies, drafts incident post-mortems, generates GitHub issues with acceptance criteria, and produces OpenAPI skeletons from plain-English requirements.
- This tool demonstrates the operational, integration, and product-decision tasks of a Technical Product Manager for streaming platforms and provides interview-ready artifacts (deployed demo, runbooks, and API specs).

3. Problem Statement / Use Case

- Streaming product teams lack a fast, repeatable way to turn operational signals and informal requests into triageable engineering work and clear API contracts; this slows incident response and partner integration.
- The target users are Technical Product Managers, SREs, and platform engineers who need quick incident summaries, reproducible runbooks, and machine-readable API contracts created from product requirements.

4. Goals and Objectives

- **Goal 1:** Implement a telemetry ingestion + KPI aggregation pipeline (startup time, p95 rebuffer, CDN errors) and surface these KPIs in a web dashboard.
- **Goal 2:** Build an agent (LangChain orchestration) that uses RAG (vector store) to draft incident post-mortems and convert plain-English requests into OpenAPI skeletons and GitHub issue payloads.
- **Goal 3:** Deliver deployable artifacts (GitHub repo, deployed demo, Postman/OpenAPI files, runbook templates, and a 5–7 minute demo video) within the 12-week course timeline.

5. Key Features / Functions

- Telemetry ingestion API to collect simulated streaming events and store raw logs.
- KPI aggregator job that computes median, p95, and error rates and exposes them via REST endpoints.
- React dashboard to visualize KPIs and surface anomaly alerts.
- Agent endpoints that: (a) summarize anomalies, (b) auto-draft post-mortems, (c) generate OpenAPI skeletons from text, and (d) create GitHub issue payloads.
- RAG capability (vector embeddings + Chroma) so the agent cites runbooks and logs when producing recommendations.

- Simple scheduler/automation to run anomaly checks and trigger agent workflows.

6. Tech Stack and Tools

- **Languages / frameworks:** Node.js (Express), React (Vite or CRA), JavaScript/TypeScript optional.
- **Datastore:** MongoDB Atlas free tier for events and KPI docs.
- **RAG / embeddings:** Chroma (local) + OpenAI or Hugging Face embeddings (OpenAI paid; HF/free options available).
- **Agent orchestration:** LangChain JS.
- **LLM provider:** OpenAI (GPT-4 — paid) or Hugging Face hosted/open models (low-cost/free).
- **Devops / hosting:** GitHub, Vercel (frontend), Render or Vercel serverless (backend), Docker for local dev.
- **Other tools:** ffmpeg (for any simulated manifest work), Postman/Swagger for API testing, Grafana or React charts for dashboards.

7. Architecture / Workflow (plain description)

- Clients (simulator or UI) POST streaming events → Express ingestion endpoint stores raw events in MongoDB.
- A scheduled KPI worker aggregates raw events into KPI documents (median, p95, error rates) and stores KPI snapshots.
- KPI snapshots are fetched by the React dashboard and by the anomaly checker; when thresholds or tail metrics breach rules, the anomaly checker queues an agent job.
- The agent (LangChain) retrieves contextual runbooks/log snippets from Chroma, calls an LLM (OpenAI/HF) with prompt templates, and returns outputs: a concise incident summary, a drafted post-mortem, a GitHub issue payload, or an OpenAPI skeleton; the UI lets users review and push issues to GitHub (or export OpenAPI files).

8. Timeline / Weekly Milestones (4-week sprint shown; this project spans 12 weeks)

> **Note:** The template below represents the initial 4-week sprint; the full project is scheduled across 12 weeks (see full schedule in project repo README).

- **Week 1 — Project Definition:** Finalize scope, select LLM provider (OpenAI vs HF), create repo, and scaffold backend + frontend skeletons.
- **Week 2 — Requirements and Goals:** Define KPI schema, acceptance criteria for agent outputs (post-mortem, OpenAPI), and required sample runbooks/log examples.
- **Week 3 — Solution Planning and Design:** Design data models, RAG indexing flow, agent prompt templates, and initial UI wireframes for dashboard + chat.
- **Week 4 — Review and Refinement:** Implement basic ingestion endpoint, run a simple KPI aggregator, test end-to-end telemetry → dashboard flow, and document sprint demo and risks.

(Full 12-week plan with week-by-week tasks and demos will be included in the repo README and course schedule; initial sprint focuses on foundations and a minimum viable demo.)

9. Risks and Risk Mitigation

- **Risk — LLM costs & rate limits:** Using commercial LLMs can be costly for repeated experiments. **Mitigation:** use small sample sizes in dev, prefer Hugging Face free models for prototyping, cache agent responses, and implement throttles.

- **Risk — Data privacy / PII in logs:** Sending raw logs to hosted LLMs may expose sensitive data. **Mitigation:** redact or synthesize logs before embedding; use local models when necessary.
- **Risk — Model hallucination / inaccurate specs:** LLM may produce incorrect OpenAPI or acceptance criteria. **Mitigation:** require human review, include unit validation (load generated OpenAPI into Swagger) and acceptance tests.
- **Risk — Infra complexity & time limits:** Setting up vector stores, transformers, and hosting can exceed the 96-hour constraint. **Mitigation:** scope the MVP tightly (use Chroma local, minimal dataset), prioritize telemetry + agent post-mortem flow first.

10. Evaluation Criteria

- **Functional correctness:** The agent must produce at least two valid post-mortem drafts from simulated incidents and one OpenAPI file that loads without syntax errors in Swagger UI.
- **Operational behavior:** Dashboard shows KPIs and triggers an anomaly alert that results in an agent-generated GitHub issue payload (or local stub).
- **Deliverables & reproducibility:** Public GitHub repo with README, Postman/OpenAPI artifacts, a deployed demo (or clear local run instructions), and a 5–7 minute demo video.
- **Interview readiness:** A one-page impact summary and 2–3 resume bullets that describe measurable results from the prototype (e.g., simulated reduction in mean time to triage).

11. Future Considerations

- For production parity: integrate real CDN logs/transcode metrics, add authenticated partner onboarding with OAuth, and replace simulated alerts with real SRE tooling (Datadog).
- Improve model reliability by adding verification layers (schema validators for OpenAPI, unit tests for generated acceptance criteria) and consider integrating LaunchDarkly or Unleash for feature flags and rollout.
- Long-term scaling: move embeddings into a managed vector DB, add CI/CD for model prompt changes, and implement role-based access for runbooks and incident outputs.

If you'd like, I will now: (A) generate the full 12-week week-by-week milestone table to attach to this document, or (B) create the starter repo skeleton (package.json, basic Express endpoints, and a minimal React UI) and add Docker Compose for local dev. Which should I produce next?